ArchSync: Evidence-Backed Conformance Checking for TypeScript Service Architectures

Anonymous Author(s)

ABSTRACT

Service changes can introduce architectural drift while preserving functional behavior, leaving maintainers without a clear link between a violated design rule and the code responsible. ArchSync combines an explicit architectural contract, provenance-aware extraction of selected TypeScript service interactions, and a Git gate that distinguishes introduced violations from changes requiring an architecture decision. We investigate implementation correctness using co-developed patches and detector signals, followed by finite rule checking and adversarial testing. All 20 development patches match their declared classifications; targeted adversarial inputs expose extraction and incremental-analysis defects that are repaired and retained as regression cases. The contribution is an executable integration of established conformance concepts with service-level evidence and change-time decisions, supported by inspectable failure and repair records. These development artifacts provide a reproducible basis for evaluating architecture checks at pull-request time.

CCS CONCEPTS

• **Software and its engineering** → **Software architecture**; *Software verification.*

KEYWORDS

architecture conformance, service-oriented systems, static analysis, architecture drift, reproducibility

ACM Reference Format:

Anonymous Author(s). 2027. ArchSync: Evidence-Backed Conformance Checking for TypeScript Service Architectures. In *12th International Conference on Intelligent Information Technology (ICIIT 2027).* 5 pages.

1 INTRODUCTION

Service-oriented systems can preserve functional behavior while introducing a forbidden dependency, losing a required interaction, or adding infrastructure that needs a design decision. These changes concern different architectural judgments. A useful check must connect an observed interaction to the declared contract, identify supporting source evidence, and distinguish a violation from an evolution. ArchSync addresses this information-extraction and decision-support problem with deterministic rules and inspectable evidence.

Our contribution is an executable integration: (1) a typed expected/observed service graph with explicit constraints; (2) provenance-aware extraction of selected TypeScript interactions and evidence; and (3) a Git-diff gate that reports introduced violations separately from evolution. Building on established model-comparison and dependency-rule concepts, the integration connects a service interaction, its supporting source location and its effect on a proposed change. Retained failure and repair records expose how extraction and incremental-analysis choices affect that decision.

We ask whether the prototype reconstructs declared graph changes and detector signals (RQ1), classifies controlled changes correctly (RQ2), supplies the declared evidence locations (RQ3), and preserves results under replay and incremental execution (RQ4). These questions guide the controlled evaluation in Section 4 and the reliability extension in Section 5. For practitioners, the Git-scoped gate provides a concrete pull-request check: introduced findings carry rule identifiers and inspectable evidence, while evolution is routed for an explicit architecture decision. The decision path executes deterministic rules without an LLM inference service.

2 BACKGROUND AND RELATED WORK

This scoped narrative review positions ArchSync against foundational and contemporary conformance approaches using sources selected purposively from the retained keyword-search corpus. Contemporary evidence is from 2022–2025; two older references establish the origins of model comparison and dependency constraints.

Problem and observation boundary. A recent mapping characterizes architecture erosion [8]. Code-review studies identify symptoms in OpenStack and OpenStack/Qt projects [7, 9]; their project overlap limits independence. Practitioner evidence identifies documentation and shared practices as ways to manage drift [1]. A recovery taxonomy and microservice-tool mapping distinguish reconstruction purposes and applicability [2, 12]. View-based drift analysis compares intended and reconstructed specifications in an industrial case [16]. These works motivate an explicit observation boundary between a source graph and the runtime architecture.

Closest conformance approaches. Software Reflexion Models establish high-level/source model comparison [11], and Dependency Constraint Language develops declarative dependency constraints [15]. ArchSync builds on these concepts. CALint checks Python dependencies against Clean Architecture rules [3]. ArchLintor detects TypeScript module-interaction violations, provides source-linked diagnostics and describes CI/CD integration [13]. ArchSync's contribution combines typed service interactions, their source evidence and Git-time introduced-finding decisions. For example, changing a fetch destination can change a service relationship while leaving module imports unchanged. Linking that relationship to a rule and a source location makes the architectural consequence inspectable; comparing base and head distinguishes an introduced violation from a new relation requiring an architecture decision. LLM-generated architectural tests introduce a further rule-translation step [10]; ArchSync executes explicitly supplied deterministic rules.

ICIIT 2027, March 4–7, 2027, Ho Chi Minh City, Vietnam
2027.

Table 1: Published context, not an executed baseline comparison. Different languages, units and datasets prevent numerical ranking.

Approach	Language / focus	Unit and decision	Evaluation evidence
ArchLintor [13]	TypeScript; module conformance	Module dependencies; allowed, forbidden, required	One author-developed eight-module example
CALint [3]	Python; Clean Architecture	Dependencies; dependency rule	Three cases in publisher abstract; full text unavailable
Recovery comparison [14]	Java/Spring; architecture recovery	Components, connections, endpoints; recovery correctness	Nine tools on 17 applications
ArchSync (this work)	TypeScript; service conformance at Git time	Typed service relations; rules and introduced findings	20 co-developed patches; 40 detector signals

Change-time feedback and verification. Repository mining studies microservice evolution [4], and industrial conformance visualization reports practitioner experience [17]. ArchSync uses relationship-level keys to keep existing findings visible while directing Git-time decisions to newly introduced findings. Evaluation-method research stresses replicability [6]; interrogation testing shows that targeted inputs can expose analyzer soundness and precision failures [5]. Retained failure/repair evidence makes the implementation of this combined workflow inspectable.

Published evidence and integration value. Table 1 compares the approaches at their reported observation units. ArchLintor illustrates module-level absence and divergence; CALint's publisher abstract reports three cases. Schneider et al. execute nine recovery tools on 17 Java/Spring applications [14], providing a design for recovery evaluation. ArchSync connects selected HTTP, PostgreSQL, Redis and AMQP observations to an explicit service contract and a base/head decision with source evidence. The useful combination is a traceable path from a changed service interaction to BLOCK or REVIEW, backed by retained regression and failure/repair records. Section 6 defines the scope of these records and the remaining comparative requirement.

3 EXECUTABLE CONTRACT AND CHANGE-TIME ANALYSIS

Let $M = (V, E, R)$ be the expected architecture, with components V, directed typed relationships E, and rules R. The observed graph O is reconstructed from supported source constructs. Component identifiers and metadata from M guide reconstruction. Edges collapse repeated observations with the same source, target and kind, while retaining their file/line evidence.

A deny rule rejects matching edges. An allow rule restricts destinations of the selected outgoing relations. A require rule demands a matching direct edge for each selected source; require-path demands a nonempty directed path, optionally restricted to one edge type. An observed rule violation takes classification precedence; otherwise a graph difference is evolution, and an unchanged graph is no-impact. The expected model remains under explicit maintainer control.

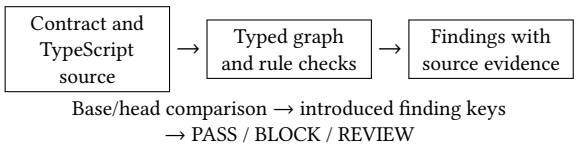


Figure 1: Git gate flow: introduced violations yield BLOCK; otherwise introduced evolution yields REVIEW, and no introduced finding yields PASS. Maintainers control the contract separately.

Tiny contract example. Let $V = \{w, a, d\}$ denote a web service, API and database, with $E = \{w \xrightarrow{\text{http}} a, a \xrightarrow{\text{data}} d\}$. Let R require a path from w to d with unrestricted edge type. An observation with edge set E satisfies the rule and is no-impact. Removing $a \to d$ violates the path requirement; adding $w \xrightarrow{\text{data}} d$ while retaining E is evolution. Against a base matching E, these edits yield BLOCK and REVIEW respectively under the gate below.

The TypeScript adapter recognizes selected global fetch calls and imported PostgreSQL, Redis and AMQP client operations. Import provenance distinguishes package clients from method-name lookalikes. HTTP targets use host identity; AMQP edges capture broker reachability. Evidence scores are heuristic. Section 6 describes the resulting observation boundary.

For a base/head pair, the gate reconstructs the base, analyzes affected source components and merges retained observations with new ones. Full-head scanning supplies a consistency oracle. New violation keys produce BLOCK, new evolution keys produce REVIEW, and no introduced finding produces PASS (Figure 1). Existing findings remain visible. Keys operate at relationship level, so an additional offending call on an existing edge need not introduce a new key. PASS denotes the absence of newly recognized gated findings. REVIEW requires a separate architecture decision; unchanged inputs remain REVIEW after a human rejection.

The expected model supplies the input contract. Call-site evidence points to observed code. A missing required relation has no absent call to localize, so it receives a deterministic component anchor. These evidence types are scored separately. Cache identity binds the baseline and configuration; later hardening also separates repository scope and analyzer version and checks payload integrity. A checksum detects accidental payload corruption.

Implementation artifacts. Core implements model validation, graph comparison and rules; Guardian implements source extraction and the Git gate. Historical measurements use pinned Core and Guardian revisions supplied in the review artifact. The packaged source archive and SHA-256 manifest separately identify the later hardening candidate.

Worked controlled example. In D1 case 06, rule `ARCH-001` forbids a direct relationship from `frontend` to `payment-service`.

Table 2: Implementation agreement with co-developed expectations on one synthetic system.

Measure	Agreement/count
D1 changed nodes / edges	5/5; 12/12
D1 classifications / violation rule sets	20/20; 7/7
Call-site cases / missing-relation anchors	9/9; 2/2
P3 decisions / exact deltas	20/20; 20/20
P3 incremental/full scan / cold–warm replay	20/20; 20/20
P3 parsed / head file instances	57/189

The patch adds `payDirectly` and a fetch call to the payment endpoint. Reconstruction adds one HTTP edge between those components; the deny rule produces an error finding with evidence at `frontend/src/app.ts`, line 14 of the patched file. Because the finding is introduced relative to the base, the Git gate returns BLOCK. A new relation without a violated rule instead produces REVIEW, reserving approval for a separate architecture decision.

4 CONTROLLED IMPLEMENTATION VERIFICATION

Objects and independence. D1 is a synthetic Order Platform baseline with five components and five relationships. Its 20 isolated patches comprise nine no-impact changes, seven violations and four evolutions. Topology labels preceded integration, but executable patches and precise evidence locations co-evolved with the analyzer. D2 has four positive and four hard-negative signals for each of five detector groups: fetch, PostgreSQL, Redis, AMQP publish and AMQP consume. It was constructed from v0.1 weaknesses and informed v0.2. P3 replays the same D1 patches through Git.

Procedure and units. D1 validates hashes, patch applicability and expected changes, then runs the baseline and each isolated patch twice (42 executions). D2 runs both pinned versions twice (four executions). P3 creates fresh baseline repositories, applies one patch, performs cold and warm checks, and separately scans the full head. Results are compared after excluding elapsed time and cache-hit state. The 20 cold/warm pairs plus full scans require 80 analyzer calls. D1 items are graph nodes/edges; D2 items are annotated detector–edge–file–line signals, which can share graph edges. Evidence matches use the declared case-level locations.

Results. Table 2 links changed nodes/edges to RQ1, classifications/rule sets to RQ2, call-site/anchor cases to RQ3, and P3 rows to RQ4, retaining the changed-item and case denominators. Full-graph totals also matched 105 nodes and 108 edges, but repeatedly include baseline structure. On D2, v0.1 yielded TP=18, FP=4, FN=2 and TN=16; v0.2 yielded TP=20, FP=0, FN=0 and TN=20. The four false positives were local PostgreSQL/AMQP-shaped receivers; the two misses were Redis operations. These targeted repairs improve the within-tool regression results. Duplicate normalized output matched, and P3 cache miss/hit results agreed in all 20 cases.

The original Windows recording used Node.js 22.16.0 and an Intel Core i7-13650HX. Cold/warm nearest-rank p50 values were 518.51/242.62 ms and p95 values 531.05/249.30 ms. They combine parsing, Git, cache I/O and reporting; RAM, power/load and filesystem-cache state were unrecorded. Historical functional replays passed Linux and Windows, checking installation and repeatability on the same inputs.

Data access. The review artifact identifies the pinned benchmark revision and input hashes. Section 6 discusses the interpretation of these development results.

5 SEPARATE TECHNICAL RELIABILITY EXTENSION

The subsequent 2026-09-26 campaign targets local development defects and preserves original failures. The preceding observations retain their original pinned analyzer packages.

Methods and AI-use disclosure. OpenAI Codex with delegated coding agents assisted in constructing the finite oracle, adversarial inputs and mutation harness, modifying the implementation, executing local checks and inspecting outputs. Reported counts come from retained executable test reports. The development process includes automated cross-agent code review and test refinement after failure inspection, as detailed below.

Finite rule checking. A separately implemented adjacency-matrix and transitive-closure oracle checks all 4,096 directed graphs on three fixed nodes, two edge types and 12 non-self edge slots. Each graph is evaluated against 192 combinations of deny/allow/require/require-path, exact/wildcard selectors and type constraints: 786,432 rule–graph evaluations. Additional checks apply ordering and consistent renaming on 241 graph states and inverse addition/removal deltas on all 4,096.

Test sensitivity. Eight declared implementation mutations target rule enforcement, path traversal and classification. The initial suite killed seven; violation demotion survived because classification was not asserted. Adding the missing assertion produced eight kills, no survivors and no execution errors in the final audit. Both rounds are retained. The two rounds document increased sensitivity to the declared fault model.

Extraction and incremental faults. Adversarial source tests exposed type-only imports treated as runtime clients, protocol mismatches, shadowed bindings and mistaken URL expressions. Binding symbols per file separates local names from package/global names. Later checks exposed wrong-host string concatenation and stale provenance after direct reassignment. Static string composition now resolves the complete URL. A further 14-case extension reproduced 11 failures caused by destructuring, unary updates and loop writes; all 43 cumulative adversarial cases pass after repair. Recognized writes conservatively invalidate the binding throughout its file, including earlier legitimate uses. Unresolved composition emits no relationship.

Incremental checks exposed missed unusual/ignored filenames, nested-component evidence loss, cross-project cache reuse and corrupt-cache behavior. Exact source ownership, Git path handling, repository/version-bound cache identity and payload checks address the reproduced cases. The extension records failing inputs, repaired outputs and remaining limitations separately. Its full local suites passed 126 Core and 315 Guardian tests on Node.js 22.16.0/macOS arm64. Two inert compiler-host callbacks are excluded from Guardian's configured coverage. The current analyzer is unreleased semantics version 0.4; it also passes a development replay of all 20 D1 cases.

6 VALIDITY AND IMPLICATIONS

Construct and internal validity. D1 and D2 are controlled, co-developed datasets; P3 reuses D1. Their perfect matches are regression-oracle agreement. D2 informed detector repairs, so its v0.1/v0.2 comparison is within-tool development evidence, not an external baseline. Repair against known inputs creates an overfitting risk. Shared assumptions between labels, implementation and the separately coded oracle can survive these checks. Handwritten positives and hard negatives emphasize anticipated constructs; their balance does not estimate the prevalence of violations in practice. File/line agreement measures specified case locations rather than complete evidence recall. The typed structural view addresses conformance, leaving responsibilities, runtime quality and practitioner usefulness for separate evaluation.

Analysis and decision boundary. Reconstruction depends on the supplied model and supported source constructs. HTTP host identity collapses port/path distinctions, and AMQP broker reachability leaves routing compatibility unresolved. Dynamic templates, alternative fallbacks, property mutation, imported-constructor rebinding, wrappers and inter-file flows can remain invisible. File-wide binding invalidation can suppress legitimate earlier uses; the extractor is neither sound nor flow-sensitive. Evidence scores are uncalibrated heuristics. Relationship-level keys can hide an additional offending call on an existing edge, and incremental/full agreement can preserve a shared omission. PASS is limited to newly recognized findings; REVIEW leaves approval and rejection enforcement to repository governance. Cache checksums address accidental corruption, not a writer able to replace both payload and checksum.

Reliability scope. The oracle enumerates a finite three-node domain, excluding self-loops, larger graphs, arbitrary wildcard fragments, extraction and component-metadata changes. Its 786,432 combinations are not independent samples. Mutation testing covers eight declared faults and includes a post-failure assertion repair; it is a test-development result rather than a whole-program mutation score. Regression counts describe executed checks. Automated cross-agent review supplies no independent human annotation, adjudication or validation signoff. Historical Linux/Windows replay and the later macOS hardening campaign have distinct execution scopes.

Expected behavior on real projects. In a TypeScript project using supported clients directly and a contract aligned with service boundaries, recognized calls should produce source-linked relations and introduced findings for review. This is an expectation from the extraction design. Wrappers, dependency injection and configuration-built endpoints can hide interactions: a missed forbidden call can falsely PASS, while an edit making a required relation unobservable can introduce a false BLOCK. Shared hosts can merge distinct destinations; file-wide invalidation after a write can suppress valid calls. An outdated contract can flag an intentional migration. Maintainers would need to inspect the evidence and update the model explicitly. Larger repositories may reduce the incremental parsing benefit when changes span many components.

External and comparative validity. No independent repository study was performed. One small synthetic topology cannot represent real dependency diversity, configuration practices, repository scale or change histories. The controlled results therefore leave field accuracy, false-block burden and developer benefit unmeasured. Repeated patches and runs are clustered within the same system. Table 1 uses different languages and observation units, precluding numerical ranking. The purposive narrative review and unavailable full texts also limit coverage; overlapping erosion-study projects limit source independence.

Conclusion validity and next steps. Finite enumeration and small changed-item counts support bounded implementation checks. Population estimates need independent study units; speedup and scalability need paired workloads with controlled timing conditions. Broader evaluation should freeze projects and tools before predictions, label the full declared source scope, retain disagreements and report per-project errors, coverage and failures. The appendix describes the prepared D3 follow-up and the separate independent study requirements. Practitioner evaluation should measure false-block burden and review effort.

7 CONCLUSION

ArchSync integrates typed service-level observations, explicit conformance rules and evidence-backed Git decisions. The controlled evaluation demonstrates agreement across 20 development patches and incremental/full replay; the reliability campaign exercises the finite rule contract and repairs reproducible extraction and cache defects. Together, these artifacts provide a reproducible foundation for architecture checks at pull-request time. Independent repository evaluation and practitioner studies are the next steps. The integration gives maintainers an inspectable path from an introduced service interaction to a rule violation or an explicit architecture decision.

REFERENCES

[1] Emilie Anthony, Astrid Berntsson, Tiziano Santilli, and Rebekka Wohlrab. 2024. We're Drifting Apart: Architectural Drift from the Developers' Perspective. In *2024 IEEE 21st International Conference on Software Architecture*. IEEE, Hyderabad, India, 101–111. https://doi.org/10.1109/ICSA59870.2024.00018

[2] Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, and Davide Taibi. 2024. Tools Reconstructing Microservice Architecture: A Systematic Mapping Study. In *Software Architecture. ECSA 2023 Tracks, Workshops, and Doctoral Symposium*. Springer Nature Switzerland, Cham, 3–18. https://doi.org/10.1007/978-3-031-66326-0_1

[3] Clevio Orlando de Oliveira Junior, Jonathan Carvalho, Fábio Fagundes Silveira, Tiago Silva da Silva, and Eduardo Martins Guerra. 2022. CALint: A Tool for Enforcing the Clean Architecture's Dependency Rule in Python. In *Computational Science and Its Applications – ICCSA 2022 Workshops: Malaga, Spain, July 4–7, 2022, Proceedings, Part V*. Springer International Publishing, Cham, 534–549. https://doi.org/10.1007/978-3-031-10548-7_39

[4] Patric Genfer and Uwe Zdun. 2024. Exploring Architectural Evolution in Microservice Systems Using Repository Mining Techniques and Static Code Analysis. In *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, September 3–6, 2024, Proceedings*. Springer Nature Switzerland, Cham, 157–173. https://doi.org/10.1007/978-3-031-70797-1_10

[5] David Kaindlstorfer, Anastasia Isychev, Valentin Wüstholz, and Maria Christakis. 2024. Interrogation Testing of Program Analyzers for Soundness and Precision Issues. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, New York, NY, USA, 319–330. https://doi.org/10.1145/3691620.3695034

[6] Marco Konersmann, Angelika Kaplan, Thomas Kühn, Robert Heinrich, Anne Koziolek, Ralf H. Reussner, Jan Jürjens, Mahmood al Doori, Nicolas Boltz, Marco Ehl, Dominik Fuchß, Katharina Großer, Sebastian Hahner, Jan Keim, Matthias Lohr, Timur Sağlam, Sophie Schulz, and Jan-Philipp Töberg. 2022. Evaluation Methods and Replicability of Software Architecture Research Objects. In *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. IEEE, Honolulu, HI, USA, 157–168. https://doi.org/10.1109/ICSA53651.2022.00023

[7] Ruiyin Li, Peng Liang, and Paris Avgeriou. 2023. Warnings: Violation symptoms indicating architecture erosion. *Information and Software Technology* 164 (2023), 107319. https://doi.org/10.1016/j.infsof.2023.107319

[8] Ruiyin Li, Peng Liang, Mohamed Soliman, and Paris Avgeriou. 2022. Understanding Software Architecture Erosion: A Systematic Mapping Study. *Journal of Software: Evolution and Process* 34, 3 (2022), e2423. https://doi.org/10.1002/smr.2423

[9] Ruiyin Li, Mohamed Soliman, Peng Liang, and Paris Avgeriou. 2022. Symptoms of Architecture Erosion in Code Reviews: A Study of Two OpenStack Projects. In *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. IEEE, 24–35. https://doi.org/10.1109/icsa53651.2022.00011

[10] Andrielly Lucena, Everton L. G. Alves, and João Brunet. 2025. Language Models as Architectural Gatekeepers: Automating Conformance Checking from Natural Language. In *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*. Sociedade Brasileira de Computação, Porto Alegre, 713–719. https://doi.org/10.5753/sbes.2025.11100

[11] Gail C. Murphy, David Notkin, and Kevin J. Sullivan. 1995. Software Reflexion Models: Bridging the Gap Between Source and High-Level Models. In *Proceedings of the Third ACM SIGSOFT Symposium on Foundations of Software Engineering*. ACM, New York, NY, USA, 18–28. https://doi.org/10.1145/222124.222136

[12] Abdul Qayum, Mengqi Zhang, Simon Colreavy, Muslim Chochlov, Jim Buckley, Dayi Lin, and Ashish Rajendra Sai. 2025. A Framework and Taxonomy for Characterizing the Applicability of Software Architecture Recovery Approaches: A Tertiary-Mapping Study. *Software: Practice and Experience* 55, 1 (2025), 100–132. https://doi.org/10.1002/spe.3364 First published online 17 July 2024.

[13] Matheus Resende, Rafael Durelli, and Ricardo Terra. 2025. ArchLintor: detecting architectural violations in TypeScript systems. In *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*. Sociedade Brasileira de Computação, Porto Alegre, 900–906. https://doi.org/10.5753/sbes.2025.11075

[14] Simon Schneider, Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, Riccardo Scandariato, and Davide Taibi. 2025. Comparison of static analysis architecture recovery tools for microservice applications. *Empirical Software Engineering* 30, 5, Article 128 (Sept. 2025). https://doi.org/10.1007/s10664-025-10686-2

[15] Ricardo Terra and Marco Tulio Valente. 2009. A Dependency Constraint Language to Manage Object-Oriented Software Architectures. *Software: Practice and Experience* 39, 12 (2009), 1073–1094. https://doi.org/10.1002/spe.931

[16] Burak Uzun and Bedir Tekinerdogan. 2024. Detecting Deviations in the Code Using Architecture View-Based Drift Analysis. *Computer Standards & Interfaces* 87 (2024), 103774. https://doi.org/10.1016/j.csi.2023.103774

[17] Filip Zamfirov, Andrei Radulescu, Jacob Krüger, and Michel R. V. Chaudron. 2025. Lessons from Visualizing Software Architecture Structure Conformance at Thermo Fisher Scientific. In *Software Engineering and Advanced Applications: 51st Euromicro Conference, SEAA 2025, Salerno, Italy, September 10–12, 2025, Proceedings, Part III*. Springer Nature Switzerland, Cham, 372–389. https://doi.org/10.1007/978-3-032-04207-1_25